



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4
по курсу «Алгоритмы компьютерной графики»
«Алгоритмы растровой развертки многоугольника»

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

- 1. Реализовать алгоритм растровой развертки многоугольника (в соответствии с вариантом): построчного сканирования многоугольника со списком активных ребер;
- 2. Реализовать алгоритм фильтрации (в соответствии с вариантом): фильтрация с взвешенным усреднением области $N \times N$, $N = 3$;
- 3. Реализовать необходимые вспомогательные алгоритмы (растеризации отрезка) с модификациями, обеспечивающими корректную работу основного алгоритма. Вспомогательный алгоритм: целочисленный алгоритм Брезенхема;
- 4. Ввод исходных данных каждого из алгоритмов производится интерактивно с помощью клавиатуры и/или мыши. Предусмотреть также возможность очистки области вывода (отмены ввода).
- 5. Растеризацию производить в специально выделенном для этого буфере в памяти с последующим копированием результата в буфер кадра OpenGL. Предусмотреть возможность изменение размеров окна.

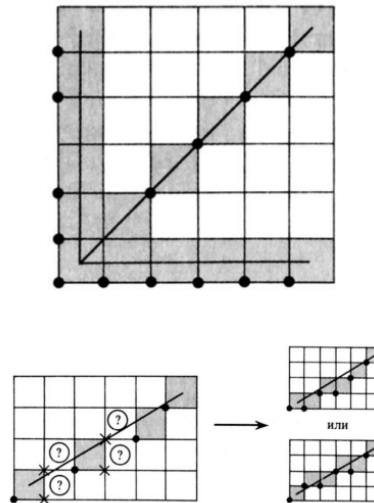
2 Основная теория

Растровые графические системы

- принцип записи изображения: построчное сканирование луча
- примитив: точка (пиксель, pixel = picture element)
- необходима процедура растеризации геометрических примитивов (растровая развертка примитивов):
 - растровая развертка в реальном времени;
 - групповое кодирование (интенсивность + длина участка);
 - клеточное кодирование (шаблоны, например, кодирование литер в алфавитно-цифровом терминале);
 - использование буфера кадра (обеспечивает промежуточное хранение изображения - растеризованных графических примитивов).

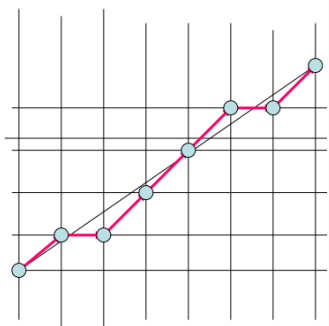
Требования к алгоритмам растровой развертки кривых

- совпадение начальной и конечной точки с заданными;
- соблюдение формы кривой;
- постоянная яркость вдоль кривой (для отрезка: независимо от длины и наклона);
- высокая производительность.



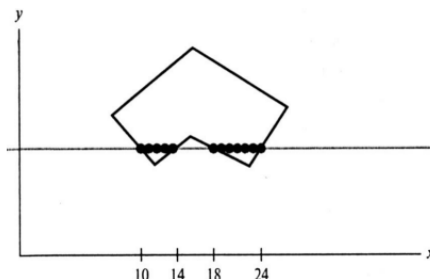
Целочисленный алгоритм Брезенхема

- $e' = 2e\Delta x$ (так как важен только знак ошибки);
- инициализация:
 - $\Delta e = 2\Delta y$
 - $e_0 = -\Delta x$
 - $x = x_0, y = y_0$
- на каждой итерации алгоритма:
 - $e \pm \Delta e$
 - $x \pm 1$,
 - $y \pm 1, e \mp 2\Delta x$, если $e \geq 0$



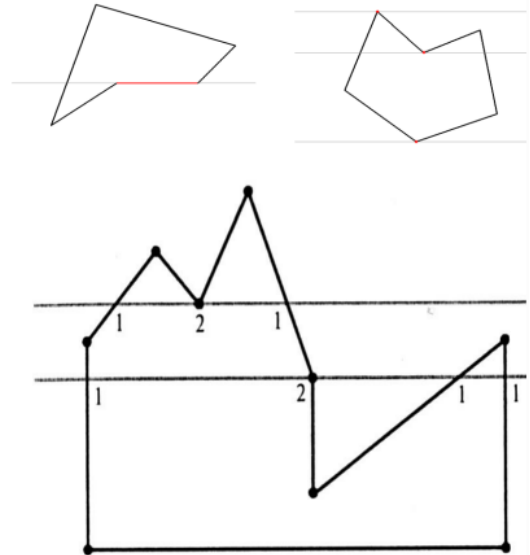
Построчное сканирование

- ребра многоугольника задаются координатами (x_1, y_1) и (x_2, y_2) его концов;
- соглашение: координата x отсчитывается слева направо, а y – сверху вниз;
- алгоритмы построчного сканирования основаны на том факте, что любое горизонтальное сечение контура многоугольника состоит из четного числа точек, следовательно, для каждой строки сканирования необходимо:
 - определить ее пересечения с ребрами многоугольника;
 - упорядочить их вдоль строки сканирования;
 - заполнить интервалы между нечетными и четными точками пересечения (использование свойств пространственной когерентности).
- достоинства алгоритма:
 - операции вывода на экран для каждого пикселя выполняются не более одного раза;
- недостатки алгоритма:
 - использование динамических структур данных (списков), что сильно усложняет код и требует дополнительной памяти.



Построчное сканирование: особые случаи определения точек пересечения строки сканирования с ребрами многоугольника

- горизонтальные ребра многоугольника - не учитываются;
- строка сканирования проходит через вершину многоугольника (можно использовать один из двух подходов):
 - смещение всех строк сканирования на $\frac{1}{2}$ пикселя (таким образом, вершины заведомо не будут лежать на строке сканирования);
 - вершина не учитывается (учитывается дважды), если оба смежных ребра лежат по одну сторону от строки сканирования, и учитывается один раз, если смежные ребра лежат по разные стороны от строки сканирования.



Построчное сканирование со списком активных ребер

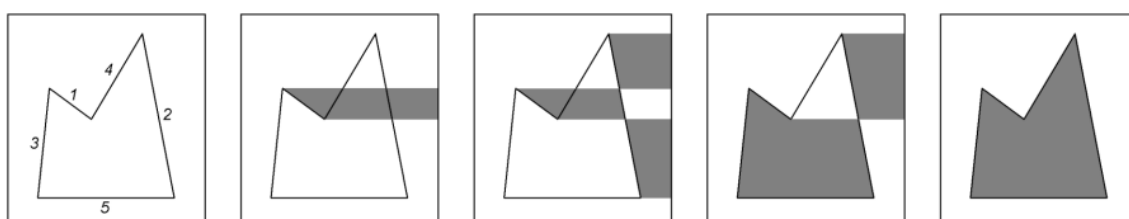
- создается у-список ребер: ребра многоугольника упорядочиваются по возрастанию по координате у начальной вершины (начальной считается вершина с наименьшей координатой у);
- на каждой итерации рассматривается только текущая строка сканирования, для которой формируется список «активных» ребер (CAP), в котором хранится информация только о ребрах многоугольника, пересекаемых текущей строкой;
- при переходе к очередной строке сканирования:
 - из CAP удаляются ребра, чья нижняя вершина оказалась выше текущей строки сканирования;
 - из у-списка добавляются ребра, начинающиеся на данной строке сканирования;
 - вычисляются новые точки пересечения строки сканирования с ребрами (используя свойство пространственной когерентности, выводятся рекуррентные соотношения):

$$\Delta x = \frac{x_2 - x_1}{y_2 - y_1}; x_{i+1} = x_i + \Delta x$$

- для каждой строки сканирования:
 - список точек пересечения с ребрами упорядочивается по возрастанию x ;
 - заполняются все промежутки вида $[x_{2i-1}; x_{2i})$.

Алгоритм заполнения многоугольника по ребрам

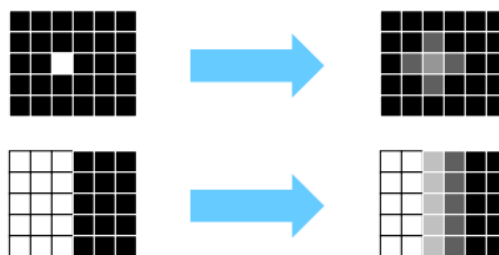
- для каждой строки сканирования инвертируются все пиксели справа от точки пересечения данной строки сканирования с ребром многоугольника;
- порядок обработки ребер многоугольника не важен;
- каждый пиксель может обрабатываться многократно.



Фильтрация

- фильтрация представляет собой свертку (convolution) сигнала (изображения) с ядром свертки (функцией фильтра) – усреднение сигнала в некоторой области;
- примеры фильтров:
 - размытие (простейшее усреднение, константное, гауссово);
 - повышение четкости;
 - нахождение границ;
 - тиснение;
 - медианный фильтр.

$$Ker[k, p] = \frac{1}{6} \cdot \begin{vmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{vmatrix}$$



$$\langle f * g \rangle (x, y) = \iint_{(\lambda, \mu) \in \Omega} f(x - \lambda)(y - \mu) \cdot g(\lambda, \mu) d\lambda d\mu$$

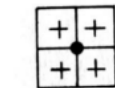
$$\langle f * g \rangle (i, j) = \sum_{l=n_0}^{n_1} \sum_{k=m_0}^{m_1} f(i - l)(j - k) \cdot g(l, k)$$

Постфильтрация

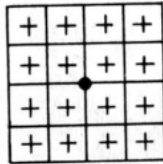
- усреднение характеристик пикселя:

- равномерное;
- взвешенное;

- Центр дисплейного пиксела
- + Центр вычисленного пиксела

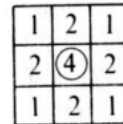


Уменьшение разрешения в 2 раза



Уменьшение разрешения в 4 раза

- Центр дисплейного пиксела

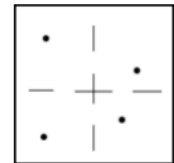
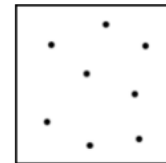
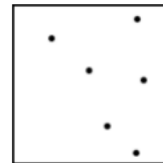


Уменьшение разрешения в 2 раза

1	2	3	4	3	2	1
2	4	6	8	6	4	2
3	6	9	12	9	6	3
4	8	12	16	12	8	4
3	6	9	12	9	6	3
2	4	6	8	6	4	2
1	2	3	4	3	2	1

Уменьшение разрешения в 4 раза

- существуют альтернативные способы выбора подпикселей



3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```

1 import glfw
2 from OpenGL.GL import *
3 import numpy as np
4
5 WIDTH, HEIGHT = 800, 600
6 polygon_points = []
7 edges = []
8 scanline_buffer = np.zeros((HEIGHT, WIDTH, 3), dtype=np.uint8)
9 drawing = True
10
11 def bresenham_line(x0, y0, x1, y1):
12     points = []
13     dx, dy = abs(x1 - x0), abs(y1 - y0)
14     sx = 1 if x0 < x1 else -1
15     sy = 1 if y0 < y1 else -1
16     err = dx - dy
17

```

```

18     while True:
19         points.append((x0, y0))
20         if x0 == x1 and y0 == y1:
21             break
22         e2 = 2 * err
23         if e2 > -dy:
24             err -= dy
25             x0 += sx
26         if e2 < dx:
27             err += dx
28             y0 += sy
29     return points
30
31 def scanline_fill():
32     global scanline_buffer
33     scanline_buffer.fill(0)
34
35     edge_table = []
36     for i in range(len(polygon_points)):
37         x0, y0 = polygon_points[i]
38         x1, y1 = polygon_points[(i + 1) % len(polygon_points)]
39
40         if y0 == y1:
41             continue
42
43         if y0 > y1:
44             x0, y0, x1, y1 = x1, y1, x0, y0
45
46         dx = (x1 - x0) / (y1 - y0)
47         edge_table.append([y0, y1, x0, dx])
48
49     edge_table.sort(key=lambda e: e[0])
50
51     active_edges = []
52     y = min(pt[1] for pt in polygon_points)
53
54     while edge_table or active_edges:
55         active_edges.extend(edge for edge in edge_table if edge[0] == y)
56         edge_table = [edge for edge in edge_table if edge[0] != y]
57
58         active_edges = [edge for edge in active_edges if edge[1] > y]
59
60         active_edges.sort(key=lambda e: e[2])
61
62         for i in range(0, len(active_edges) - 1, 2):
63             x_start = int(active_edges[i][2])

```

```

64         x_end = int(active_edges[i + 1][2])
65         scanline_buffer[y, x_start:x_end + 1] = (255, 255, 255)
66
67     y += 1
68     for edge in active_edges:
69         edge[2] += edge[3] # x += dx
70
71 def apply_filter():
72     global scanline_buffer
73     kernel = np.array([[1, 2, 1],
74                        [2, 4, 2],
75                        [1, 2, 1]]) / 16.0
76     height, width, _ = scanline_buffer.shape
77     new_buffer = np.copy(scanline_buffer)
78
79     for y in range(1, height - 1):
80         for x in range(1, width - 1):
81             region = scanline_buffer[y - 1:y + 2, x - 1:x + 2, 0]
82             new_buffer[y, x] = np.sum(region * kernel)
83
84     scanline_buffer = new_buffer
85
86 def draw_buffer():
87     glDrawPixels(WIDTH, HEIGHT, GL_RGB, GL_UNSIGNED_BYTE,
88                 scanline_buffer)
89
90 def mouse_button_callback(window, button, action, mods):
91     global drawing
92     if action == glfw.PRESS and button == glfw.MOUSE_BUTTON_LEFT:
93
94         width, height = glfw.get_framebuffer_size(window)
95
96         x, y = glfw.get_cursor_pos(window)
97
98         x = int(x * WIDTH / width)
99         y = int(y * HEIGHT / height)
100        polygon_points.append((x, y))
101        print("[%d, %d]" % (x, y))
102
103 def framebuffer_size_callback(window, width, height):
104     global WIDTH, HEIGHT, scanline_buffer
105
106     WIDTH, HEIGHT = width, height
107
108     glViewport(0, 0, WIDTH, HEIGHT)

```



```

109     scanline_buffer = np.zeros((HEIGHT, WIDTH, 3), dtype=np.uint8)
110
111     glMatrixMode(GL_PROJECTION)
112     glLoadIdentity()
113     glOrtho(0, WIDTH, 0, HEIGHT, -1, 1)
114     glMatrixMode(GL_MODELVIEW)
115
116
117 def key_callback(window, key, scancode, action, mods):
118     global drawing, scanline_buffer
119     if action == glfw.PRESS:
120         if key == glfw.KEY_ENTER:
121             drawing = False
122             scanline_fill()
123         elif key == glfw.KEY_F:
124             apply_filter()
125         elif key == glfw.KEY_C:
126             polygon_points.clear()
127             edges.clear()
128             scanline_buffer.fill(0)
129             drawing = True
130
131 def main():
132     if not glfw.init():
133         return
134
135     window = glfw.create_window(WIDTH, HEIGHT, "Lab4", None, None)
136     if not window:
137         glfw.terminate()
138         return
139
140     glfw.make_context_current(window)
141     glfw.set_mouse_button_callback(window, mouse_button_callback)
142     glfw.set_key_callback(window, key_callback)
143     glfw.set_framebuffer_size_callback(window, framebuffer_size_callback
144 )
145
146     glPixelZoom(1, -1)
147     glRasterPos2f(-1, 1)
148
149     while not glfw.window_should_close(window):
150         glClear(GL_COLOR_BUFFER_BIT)
151
152         if drawing:
153             glBegin(GL_LINE_LOOP)
154             glColor3f(1, 1, 1)

```

```

154         for x, y in polygon_points:
155             glVertex2f(2 * x / WIDTH - 1, 2 * y / HEIGHT - 1)
156         glEnd()
157
158     draw_buffer()
159     glfw.swap_buffers(window)
160     glfw.poll_events()
161
162     glfw.terminate()
163
164 if __name__ == "__main__":
165     main()

```

Результат запуска представлен на рисунках 1–2.

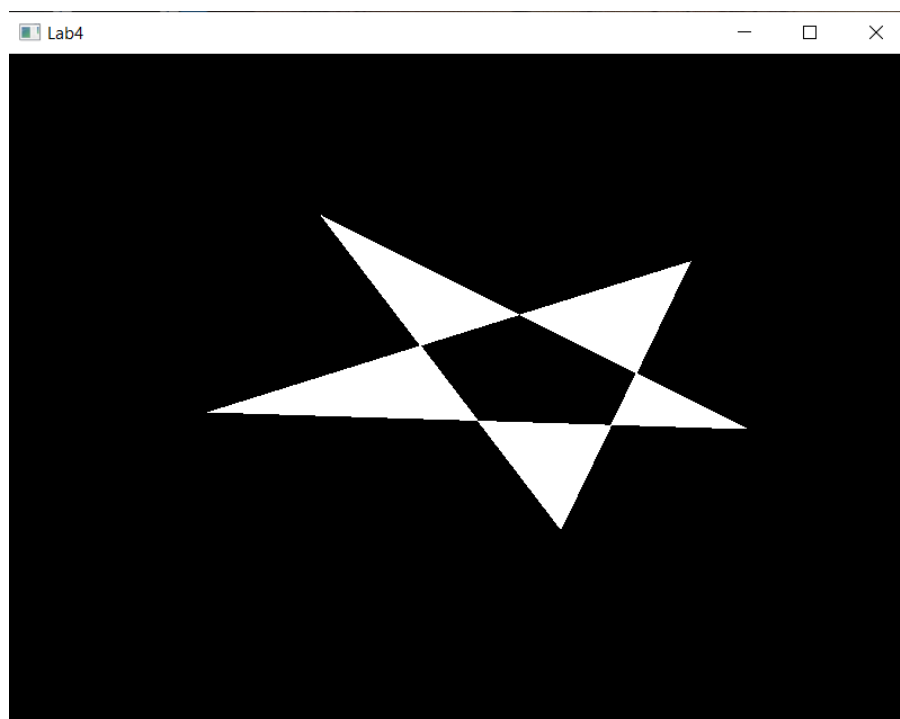


Рис. 1 — Пример фигуры

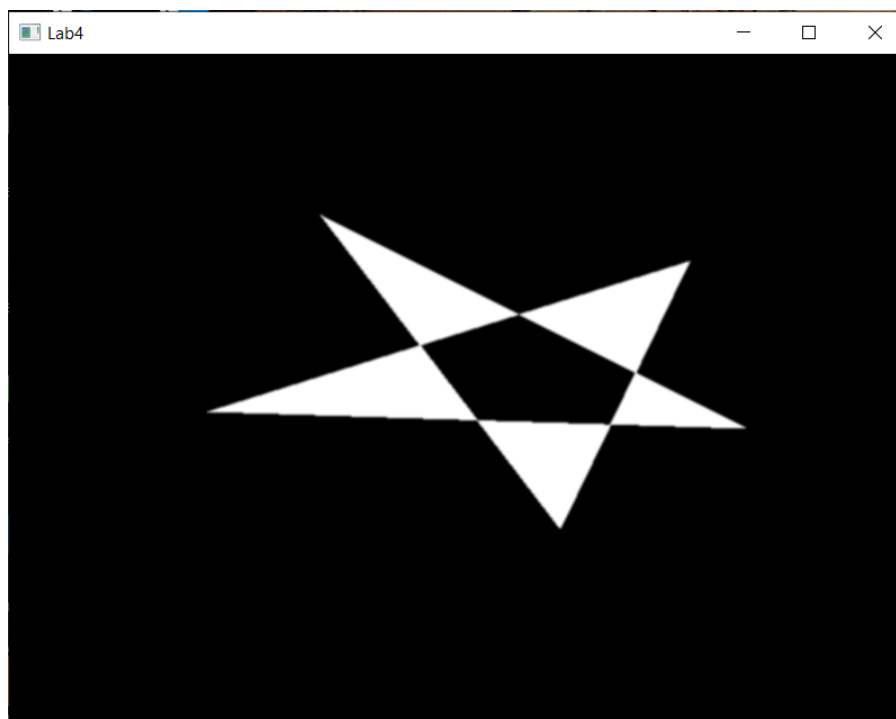


Рис. 2 — Пример фигуры с применением фильтрации

4 Заключение

В данной работе был реализован алгоритм построчного сканирования многоугольника со списком активных рёбер, который обеспечивает корректную и эффективную растеризацию произвольных многоугольных областей. Для построения границ использовался целочисленный алгоритм Брезенхема, позволяющий точно определять пиксели, принадлежащие рёбрам. Закраска выполнялась в специально выделенном буфере памяти, а затем результат копировался в буфер кадра OpenGL, что соответствует заданным требованиям.

Кроме того, была реализована фильтрация изображения методом взвешенного усреднения на основе маски 3×3 , что позволило добиться сглаживания границ и устранения резких переходов. Алгоритмы были интегрированы в интерактивную систему, обеспечивающую ввод данных с помощью мыши и клавиатуры. Также была предусмотрена возможность очистки области вывода и повторного ввода данных, что делает программу удобной в использовании.

Итоговая реализация демонстрирует стабильную работу алгоритмов и соответствует поставленной задаче.